

# RISC-V Processor Design in iVerilog

## Introduction to Processor Architecture

Spring 2026

Team 33

Gauri Krishnan  
2024102073

Dhanika Kothari  
2024102011

Moksha Choksi  
2024102048

## Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>3</b>  |
| <b>2</b> | <b>Sequential Datapath Architecture</b>               | <b>3</b>  |
| 2.1      | Program Counter (PC)                                  | 4         |
| 2.2      | Instruction Memory                                    | 4         |
| 2.3      | Register File                                         | 5         |
| 2.4      | Control Unit                                          | 6         |
| 2.5      | Immediate Generation                                  | 7         |
| 2.6      | ALU Control                                           | 7         |
| 2.7      | Arithmetic Logic Unit (ALU)                           | 8         |
| 2.8      | Data Memory                                           | 8         |
| 2.9      | Multiplexers                                          | 9         |
| 2.10     | Adder Blocks                                          | 9         |
| <b>3</b> | <b>The Sequential Processor</b>                       | <b>9</b>  |
| <b>4</b> | <b>Testing and Verification</b>                       | <b>9</b>  |
| 4.1      | Sum of 1st N numbers                                  | 9         |
| 4.2      | Fibonacci Sequence                                    | 10        |
| <b>5</b> | <b>Challenges faced</b>                               | <b>11</b> |
| <b>6</b> | <b>Introduction to Pipelining</b>                     | <b>11</b> |
| 6.1      | Instruction Fetch (IF)                                | 12        |
| 6.2      | Instruction Decode (ID)                               | 12        |
| 6.3      | Execute (EX)                                          | 12        |
| 6.4      | Memory Access (MEM)                                   | 12        |
| 6.5      | Write Back (WB)                                       | 12        |
| <b>7</b> | <b>Pipeline Registers</b>                             | <b>13</b> |
| <b>8</b> | <b>Data Hazards and Forwarding</b>                    | <b>13</b> |
| 8.1      | Forwarding Unit                                       | 13        |
| 8.2      | Hazard Detection Unit                                 | 14        |
| 8.2.1    | Load-Use Data Hazard                                  | 14        |
| 8.2.2    | Control Hazard                                        | 15        |
| <b>9</b> | <b>Testing and Verification of Pipeline Processor</b> | <b>16</b> |
| 9.1      | Forwarding                                            | 17        |
| 9.2      | Stall                                                 | 17        |
| 9.3      | Flush                                                 | 18        |

|                           |           |
|---------------------------|-----------|
| <b>10 Contributions</b>   | <b>18</b> |
| 10.1 Sequential . . . . . | 18        |
| 10.2 Pipelining . . . . . | 18        |

# 1 Introduction

This project focuses on the design and implementation of a Sequential (Non-Pipelined) RISC-V processor based on the RV64I base integer instruction set. The processor was implemented using Verilog HDL and simulated using iVerilog.

The sequential processor executes one complete instruction per clock cycle, meaning that the next instruction begins execution only after the current instruction has fully completed. The supported instructions include:

- Arithmetic: add, sub, addi
- Logical: and, or
- Memory: ld, sd
- Control Flow: beq

The architecture strictly follows a modular design approach, where each component of the datapath was designed, verified independently, and later integrated into the complete processor.

The processor follows the Big-Endian memory organization as specified in the project document.

## 2 Sequential Datapath Architecture

The sequential processor executes instructions in five logical stages:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

However, unlike pipelined processors, these stages occur sequentially within a single clock cycle. The total clock period must satisfy:

$$T_{clock} \geq T_{IF} + T_{ID} + T_{EX} + T_{MEM} + T_{WB}$$

Thus, the clock frequency is limited by the slowest cumulative path in the datapath.

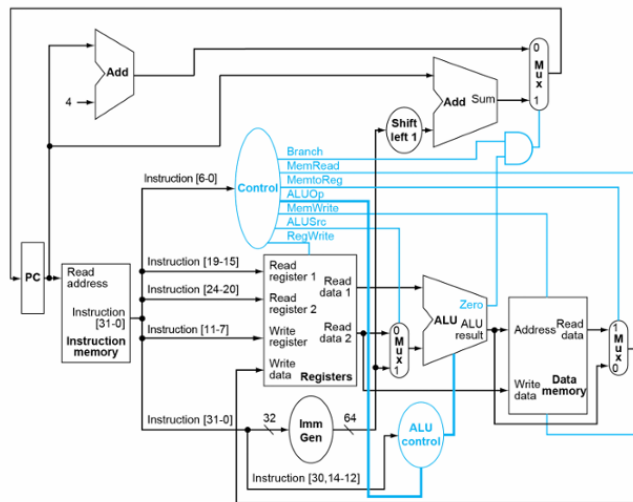


Figure 1: Complete Datapath of the Sequential Processor

## 2.1 Program Counter (PC)

The Program Counter consists of a 64-bit register that stores the address of the current instruction being executed.

**Inputs:** clk, reset, pc.in

**Outputs:** pc.out

The Program Counter is updated on every positive edge of the clock signal. When the reset signal is asserted, the PC is cleared to zero. Otherwise, the value present at pc.in is loaded into the PC.

During normal sequential execution, the PC is incremented by 4 bytes to fetch the next instruction, since each instruction in RV64I is 32 bits (4 bytes) long and memory is byte-addressable. In case of branch or instructions, the PC is updated with the computed branch target address instead of PC + 4.

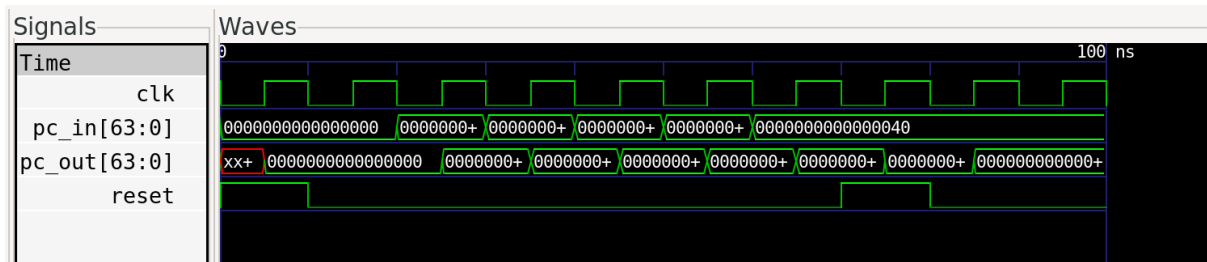


Figure 2: Output of PC file Testbench

## 2.2 Instruction Memory

The Instruction Memory stores the program instructions fetched by the processor during execution. It is implemented as a byte-addressable memory array and outputs a 32-bit instruction corresponding to the given address.

**Inputs:** clk, reset, address

**Outputs:** instruction

A macro IMEM\_SIZE defines the total size of the memory block. In this design, a size of 4096 bytes (or greater) is used, which is sufficient for typical program storage.

The instructions are loaded from an external file `instructions.txt` located in the same directory as the Verilog source file. The file contains hexadecimal byte values, with one byte per line. Every four consecutive lines together represent one complete 32-bit instruction.

During simulation initialization, the file is opened and parsed line-by-line. Each byte is stored sequentially into the instruction memory array.

When an address is applied at the input `addr`, the corresponding 32-bit instruction is formed by concatenating four consecutive bytes:

$$instr = \{memory[addr], memory[addr + 1], memory[addr + 2], memory[addr + 3]\}$$

Since each instruction is 32 bits (4 bytes) long, instruction addresses increment by 4 during sequential execution. If the address exceeds the valid memory range, a default value of 32'b0 is returned.

The instruction memory operates as a read-only memory during execution and does not require a write-enable signal.

```

● moksha@LAPTOP-4EPBCM0Q:~/IPA/mpiwithtb/mpi$ iverilog -o instruction_mem_tb instruction_mem.v instruction_mem_tb.v
● moksha@LAPTOP-4EPBCM0Q:~/IPA/mpiwithtb/mpi$ vvp instruction_mem_tb
VCD info: dumpfile instruction_mem_tb.vcd opened for output.
Test 1: Read first instruction
Address = 0000000000000000
Instruction = 00500113 (Expected: 00500113)
Result: PASS

Test 2: Read second instruction
Address = 0000000000000004
Instruction = 00a00193 (Expected: 00a00193)
Result: PASS

Test 3: Boundary check (last valid instruction)
Address = 0000000000000ffc
Instruction = 00000000
Result: PASS

Test 4: Out-of-range address
Address = 0000000000001000
Instruction = 00000000 (Expected: 00000000)
Result: PASS

=====
Total Tests : 4
Passed      : 4
Failed      : 0
=====
instruction_mem_tb.v:120: $finish called at 60000 (1ps)

```

Figure 3: Output for Instruction Memory file Testbench

## 2.3 Register File

The Register File consists of 32 general-purpose registers, each 64 bits wide.

**Inputs:** clk, reset, read\_reg1, read\_reg2, write\_reg, write\_data, reg\_write\_en

**Outputs:** read\_data1, read\_data2

The register file allows simultaneous reading of two registers and writing to one register in a single clock cycle. The write operation occurs only when reg\_write\_en is asserted. Register x0 is hardwired to zero and cannot be modified. After execution completes, the contents of all 32 registers are written to `register file.txt` in hexadecimal format.

```

Test 1: Reset verification (all registers zero)
x10 = 0000000000000000 (Expected: 0000000000000000)
Result: PASS

Test 2: Write maximum positive 64-bit value to x5
x5 = 7fffffffffffffff (Expected: 7FFFFFFFFFFFFFFF)
Result: PASS

Test 3: Write negative number to x8
x8 = 18446744073709539271 (Expected: -12345)
Result: PASS

Test 4: Overwrite x5
x5 = 111122233334444 (Expected: 111122233334444)
Result: PASS

Test 5: Simultaneous read of x5 and x8
x5 = 111122233334444
x8 = 18446744073709539271
Result: PASS

=====
Total Tests : 5
Passed      : 5
Failed      : 0
=====
regfile_tb.v:119: $finish called at 65000 (1ps)

```

Figure 4: Output for Register File Testbench

## 2.4 Control Unit

The Control Unit decodes the opcode field of the instruction and generates the control signals required for proper operation of the datapath.

**Input:** opcode

**Outputs:** Branch, MemRead, MemtoReg, ALUOp[1:0], MemWrite, ALUSrc, RegWrite

Based on the instruction type (R-type, I-type, S-type, or B-type), the Control Unit produces the following signals:

- **RegWrite** is asserted for instructions that write a result back to the register file, such as R-type instructions, `addi`, and `ld`. It remains de-asserted for `sd` and `beq`.
- **ALUSrc** selects the second ALU operand. It is set to 1 for instructions using an immediate value (e.g., `addi`, `ld`, `sd`) and set to 0 when both operands come from registers (R-type, `beq`).
- **MemRead** is asserted only for load instructions (`ld`) to enable reading from data memory.
- **MemWrite** is asserted only for store instructions (`sd`) to allow writing to data memory.
- **MemtoReg** determines the source of data written back to the register file. It is set to 1 for load instructions so that memory data is written back, and set to 0 for arithmetic instructions so that the ALU result is written.
- **Branch** is asserted for branch instructions (`beq`). Together with the zero flag from the ALU, it determines whether the PC should update to the branch target address.
- **ALUOp** provides a high-level classification of the instruction to the ALU Control block. One encoding corresponds to R-type operations, another to branch comparisons, and another to immediate-based arithmetic or address calculation.

An indirect control signal is generated by logically ANDing the Branch signal with the zero flag. When asserted, the PC is updated with the branch target address; otherwise, it increments by 4 for sequential execution.

```
Expected -> ALUOp=10, RegWrite=1
Actual   -> ALUOp=10, RegWrite=1

Result : PASS

Test 2 : I-Type (addi)
Expected -> ALUOp=00, ALUSrc=1, RegWrite=1
Actual   -> ALUOp=00, ALUSrc=1, RegWrite=1

Result : PASS

Test 3 : Load (ld)
Expected -> ALUOp=00, MemRead=1, MemtoReg=1
Actual   -> ALUOp=00, MemRead=1, MemtoReg=1

Result : PASS

Test 4 : Store (sd)
Expected -> ALUOp=00, MemWrite=1
Actual   -> ALUOp=00, MemWrite=1

Result : PASS

Test 5 : Branch (beq)
Expected -> ALUOp=01, Branch=1
Actual   -> ALUOp=01, Branch=1

Result : PASS

Test 6 : Undefined Opcode
Expected -> ALUOp=00, All control signals 0
Actual   -> ALUOp=00, Branch=0, RegWrite=0

Result : PASS

=====
Control Unit Verification Summary
Total Cases : 6
Successful  : 6
Unsuccessful: 0
=====
control tb.v:123: $finish called at 30000 (1ps)
```

Figure 5: Output for Control Unit Testbench

## 2.5 Immediate Generation

The Immediate Generation block extracts and sign-extends the immediate field from the 32-bit instruction.

**Input:** instruction (32-bit)

**Output:** immediate (64-bit signed extended)

The immediate is calculated according to the Instruction type as follows:

- For I-type instructions such as addi and ld, the immediate is extracted from bits [31:20].
- For S-type instructions such as sd, the immediate is formed by combining bits [31:25] and [11:7].
- For B-type instructions such as beq, the immediate is constructed using bits [31], [7], [30:25], and [11:8], followed by a left shift of one bit to ensure word alignment. The extracted value is then sign-extended to 64 bits.

|              |    |    |    |     |    |     |    |        |    |             |   |        |   |        |
|--------------|----|----|----|-----|----|-----|----|--------|----|-------------|---|--------|---|--------|
| 31           | 27 | 26 | 25 | 24  | 20 | 19  | 15 | 14     | 12 | 11          | 7 | 6      | 0 |        |
| funct7       |    |    |    | rs2 |    | rs1 |    | funct3 |    | rd          |   | opcode |   | R-type |
| imm[11:0]    |    |    |    |     |    | rs1 |    | funct3 |    | rd          |   | opcode |   | I-type |
| imm[11:5]    |    |    |    | rs2 |    | rs1 |    | funct3 |    | imm[4:0]    |   | opcode |   | S-type |
| imm[12:10:5] |    |    |    | rs2 |    | rs1 |    | funct3 |    | imm[4:1 11] |   | opcode |   | B-type |

Figure 6: Immediate Reconstruction for each Instruction type

```

Test 1 : I-Type (addi) Positive Immediate
Immediate = 25 | Expected = 25 --> PASS

Test 2 : I-Type (addi) Negative Immediate
Immediate = 18446744073709551600 | Expected = 18446744073709551600 --> PASS

Test 3 : S-Type (sd)
Immediate = 8 | Expected = 8 --> PASS

Test 4 : B-Type (beq) Positive Offset
Immediate = 16 | Expected = 16 --> PASS

Test 5 : B-Type (beq) Negative Offset
Immediate = 18446744073709551608 | Expected = 18446744073709551608 --> PASS

=====
Immediate Generator Verification
Total Tests : 5
Passed      : 5
Failed      : 0
=====
imm_gen_tb.v:106: $finish called at 25000 (1ps)

```

Figure 7: Output for Immediate Generator Testbench

## 2.6 ALU Control

The ALU Control block determines the exact ALU operation to be performed.

**Inputs:** ALUOp[1:0], funct3, funct7

**Output:** ALUControl (4-bit)

The ALUOp signal from the Control Unit indicates the general instruction type, while the funct3 and funct7 fields further specify the exact operation for R-type instructions. Based on these inputs, the ALUControl signal selects the required arithmetic or logical function such as addition, subtraction, AND, or OR.

```

===== Test 1 : ALUOp=00 (Load/Store/Addi) =====
Expected = 0000 | Actual = 0000 | PASS

===== Test 2 : ALUOp=01 (Branch) =====
Expected = 1000 | Actual = 1000 | PASS

===== Test 3 : R-Type ADD =====
Expected = 0000 | Actual = 0000 | PASS

===== Test 4 : R-Type SUB =====
Expected = 1000 | Actual = 1000 | PASS

===== Test 5 : R-Type AND =====
Expected = 0111 | Actual = 0111 | PASS

===== Test 6 : R-Type OR =====
Expected = 0110 | Actual = 0110 | PASS

===== Test 7 : Invalid funct3 =====
Expected = 1111 | Actual = 1111 | PASS

===== Test 8 : Invalid ALUOp =====
Expected = 1111 | Actual = 1111 | PASS

=====
ALU Control Verification Summary
Total Cases : 8
Successful   : 8
Unsuccessful: 0
=====
alu_control_tb.v:97: $finish called at 40000 (1ps)

```

Figure 8: Output for ALU Control Testbench

## 2.7 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit performs arithmetic and logical operations on two 64-bit operands.

**Inputs:** input1 (64-bit), input2 (64-bit), control

**Outputs:** result (64-bit), zero flag

Depending on the control signal, the ALU performs operations such as addition, subtraction, AND, and OR. The zero flag is asserted when the result is equal to zero, which is used in evaluating branch conditions.

## 2.8 Data Memory

The Data Memory is a byte-addressed memory block of size 1024 bytes.

**Inputs:** clk, reset, address, write\_data, MemRead, MemWrite

**Output:** read\_data (64-bit) When MemWrite is asserted, the 64-bit value present on write\_data is

written to the specified memory address. When MemRead is asserted, the data stored at the specified address is returned on read\_data. The memory has a latency of one clock cycle, meaning the output data becomes available at the next positive clock edge. The contents of data memory are written to `data mem.txt` after execution.



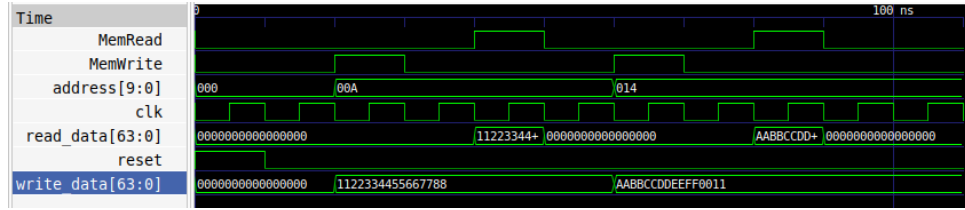


Figure 9: GTK-Wave Output for Data Memory Testbench

## 2.9 Multiplexers

Multiplexers are used at multiple points in the datapath to select between alternative inputs.

**Inputs and Outputs:** Depend on placement in datapath as follows:

- One multiplexer selects between register data and immediate value for the second ALU operand, controlled by ALUSrc.
- Another multiplexer selects between the ALU result and data memory output during the write-back stage, controlled by MemtoReg.
- A third multiplexer selects between  $PC + 4$  and the branch target address, controlled by the logical AND of the Branch signal and the zero flag.

## 2.10 Adder Blocks

Two adder blocks are used in the datapath.

**Inputs and Outputs:** 64-bit operands, 64-bit sum

One adder computes  $PC + 4$  for sequential execution. The other adder computes the branch target address by adding the sign-extended immediate to the current PC value.

## 3 The Sequential Processor

The individual modules were integrated into a wrapper file to form the complete Sequential Processor. The processor executes one instruction per clock cycle and runs for an extra clock cycle to detect the end of the program. This implementation supports the following RISC-V instructions: addi, and, sub, add, or, ld, sd, and beq.

## 4 Testing and Verification

Each individual module was tested using verilog testbenches and the final Sequential Processor was tested using assembly code that was converted to machine language instructions.

The assembly code and the machine language instructions are saved as `name_instructions_exp.txt` and `name_instructions.txt` respectively.

The final output is stored in `register_file.txt` and contains the 32 64-bit register values in hexadecimal, followed by the number of clock cycles in decimal.

### 4.1 Sum of 1st N numbers

Assembly code:

```

1 addi x1, x0, 30 //value of N
2 addi x2, x0, 0
3 addi x3, x0, 1
4 add x10, x3, x0
5 add x2, x2, x3
6 addi x3, x3, 1
7 sub x4, x3, x1

```

```

8 beq x4, x10, 8
9 beq x0, x0, -16

```

Output in register\_file.txt:

```

x0: 0000000000000000
x1: 0000000000000001e // N = 30
x2: 0000000000000001d1 // sum
x3: 0000000000000001f // value of i after exit
x4: 00000000000000001
x5: 0000000000000000
x6: 0000000000000000
x7: 0000000000000000
x8: 0000000000000000
x9: 0000000000000000
x10: 0000000000000001
Cycle count: 154

```

## 4.2 Fibonacci Sequence

Assembly code:

```

1 addi x1, x0, 8
2 addi x2, x0, 0
3 addi x3, x0, 0
4 addi x4, x0, 1
5
6 sd x3, 0(x2)
7 addi x2, x2, 8
8 sd x4, 0(x2)
9 addi x2, x2, 8
10
11 addi x5, x0, 2
12 beq x5, x1, 32
13
14 loop:
15 add x6, x3, x4
16 sd x6, 0(x2)
17 addi x2, x2, 8
18 addi x5, x5, 1
19 add x3, x0, x4
20 add x4, x0, x6
21 beq x5, x1, 12
22 beq x0, x0, -28
23
24 //load
25 addi x2, x0, 0
26 ld x10, 0(x2)
27 ld x11, 8(x2)
28 ld x12, 16(x2)
29 ld x13, 24(x2)
30 ld x14, 32(x2)
31 ld x15, 40(x2)
32 ld x16, 48(x2)
33 ld x17, 56(x2)

```

Output in register\_file.txt:

```

x0 : 0000000000000000
x1 : 0000000000000008 // loop limit (no. of terms)
x2 : 0000000000000000

```

```

x3  : 00000000000000008
x4  : 0000000000000000d
x5  : 00000000000000008 // value of i after exit
x6  : 0000000000000000d // last computed value
x7  : 00000000000000000
x8  : 00000000000000000
x9  : 00000000000000000
x10 : 00000000000000000 //0
x11 : 00000000000000001 //1
x12 : 00000000000000001 //1
x13 : 00000000000000002 //2
x14 : 00000000000000003 //3
x15 : 00000000000000005 //5
x16 : 00000000000000008 //8
x17 : 0000000000000000d //13
Cycle count: 70

```

## 5 Challenges faced

- Consistent implementation of Big-endian memory format throughout the project
- Implementing accurate immediate reconstruction according to each Instruction type
- Proper byte addressing for 64-bit load/store operations
- Correct handling of branch termination condition
- Stopping execution exactly when a 0 instruction is fetched without missing or adding extra cycles.

## 6 Introduction to Pipelining

Modern processors aim to improve performance by executing multiple instructions concurrently. One widely used technique to achieve this is **pipelining**. Instruction pipelining divides the execution of an instruction into several stages and allows multiple instructions to be processed simultaneously, with each instruction occupying a different stage of the pipeline.

In a non-pipelined processor, each instruction must complete all stages of execution before the next instruction begins. This results in inefficient utilization of hardware resources and increased execution time. In contrast, pipelining overlaps the execution of instructions, thereby increasing instruction throughput.

The idea of pipelining is similar to an assembly line in manufacturing. While one instruction is being decoded, another instruction can be fetched from memory, and yet another can be executed. Although the latency of a single instruction may not decrease significantly, the overall number of instructions completed per unit time increases.

The processor implemented in this project uses a **five-stage instruction pipeline**, which is commonly used in RISC architectures such as RISC-V and MIPS. The five stages are:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

Each stage performs a specific part of the instruction execution process, and pipeline registers are used between stages to store intermediate results.

Pipeline registers are placed between stages to store intermediate results and control signals.

## 6.1 Instruction Fetch (IF)

The IF stage fetches the instruction from instruction memory using the **Program Counter (PC)**.

### Operations:

- PC provides the instruction address
- Instruction memory returns the instruction
- PC is updated to the next instruction ( $PC + 4$ )

**Output signals:** - Instruction, PC.

## 6.2 Instruction Decode (ID)

The ID stage decodes the instruction and reads operand values from the register file.

### Operations:

- Extract fields: opcode, rs1, rs2, rd
- Read operands from register file
- Generate control signals using the Control Unit
- Generate immediate value using the Immediate Generator

**Output signals:** - Register values (rs1, rs2), Immediate value, Control signals.

## 6.3 Execute (EX)

The EX stage performs arithmetic and logical operations using the ALU.

### Operations:

- ALU performs operations such as ADD, SUB, AND, OR
- Computes memory address for LD and SD
- Evaluates branch conditions for BEQ

**Output signals:** - ALU result, Zero flag, Branch target address.

## 6.4 Memory Access (MEM)

The MEM stage accesses the data memory for load and store instructions.

### Operations:

- Load (LD): read data from memory
- Store (SD): write register value to memory
- For non-memory instructions, the ALU result is forwarded

**Output signals:** - Memory read data, ALU result.

## 6.5 Write Back (WB)

The WB stage writes the final result back to the register file.

### Operations:

- Select result using MemtoReg multiplexer
- Write result into destination register rd

The value written back may be either: ALU result (arithmetic instructions), Memory data (load instructions)

## 7 Pipeline Registers

Pipeline registers are used to separate the stages of the processor pipeline and store intermediate data and control signals between consecutive stages. They ensure that each instruction progresses through the pipeline correctly while multiple instructions are executed simultaneously.

In a 5-stage pipelined processor, four pipeline registers are used. Each register stores both **data signals** and **control signals** required by the next stage.

- IF/ID - Instruction, Program Counter
- ID/EX - Register operands (**rs1**, **rs2**), Immediate value, Destination register (**rd**), ALUOp, ALUSrc, Branch, MemRead, MemWrite, MemtoReg, RegWrite
- EX/MEM - ALU result, Branch target address, Zero flag, Store data (value from **rs2**), MemRead, MemWrite, MemtoReg, RegWrite, Destination register (**rd**)
- MEM/WB - Data read from memory, ALU result, Destination register (**rd**), MemtoReg, RegWrite

## 8 Data Hazards and Forwarding

### 8.1 Forwarding Unit

Some data hazards can be solved by **forwarding** (also called *bypassing*) a result from the MEM or WB stage to a dependent instruction in the EX stage. This requires additional hardware, specifically, multiplexers in front of the ALU to select its operands from the register file, or from the EX/MEM or MEM/WB pipeline registers.

#### Forwarding Cases

1. **EX Hazard:** The instruction immediately before the current one writes to a register that the current instruction reads. In this case the result is available in the EX/MEM pipeline register and is forwarded directly to the EX stage. For example:

```
1  addi x5, x0, 10
2  add  x6, x5, x7
```

Here **x5** is written by the first instruction and read by the second. The same scenario arises if the second instruction is a store, a load, or a branch that reads **x5**.

2. **MEM Hazard:** A value produced by an earlier instruction is required two instructions later, before it has been written back to the register file. In this case, the result is forwarded from the MEM/WB pipeline register directly to the EX stage. For example:

```
1  addi x3, x0, 5
2  addi x8, x0, 20
3  sub  x9, x3, x8
```

Here, the third instruction needs the value of **x3** generated by the first instruction. Since the write-back has not yet occurred, the value is forwarded from the MEM/WB stage to the EX stage.

#### Inputs and Outputs

The forwarding unit is a purely combinational module. Its interface is as follows:

- **Inputs:**
  - **rs1\_idx[4:0]**, **rs2\_idx[4:0]** — source register specifiers from the ID/EX pipeline register
  - **rd\_exmem[4:0]** — destination register from the EX/MEM pipeline register
  - **rd\_memwb[4:0]** — destination register from the MEM/WB pipeline register
  - **regwrite\_exmem** — RegWrite control signal from EX/MEM
  - **regwrite\_memwb** — RegWrite control signal from MEM/WB

- **Outputs:**

- `forwardA[1:0]` — selects the first ALU operand (for `rs1`)
- `forwardB[1:0]` — selects the second ALU operand (for `rs2`)

The encoding of the forwarding signals is: `2'b00` = no forwarding (use register file), `2'b10` = forward from EX/MEM, `2'b01` = forward from MEM/WB.

## Forwarding Logic

The forwarding conditions implemented in `forwarding_unit.v` are as follows.

**EX Hazard** - forward from EX/MEM to the EX stage:

```
1 if (regwrite_exmem && (rd_exmem != 0) && (rd_exmem == rs1_idx))
2     forwardA = 2'b10;
3 if (regwrite_exmem && (rd_exmem != 0) && (rd_exmem == rs2_idx))
4     forwardB = 2'b10;
```

The EX hazard check verifies that the destination register `rd` in the EX/MEM stage matches the source register `rs1` or `rs2` in the ID/EX stage, while ensuring that a write is occurring and that the destination is not `x0`.

**MEM Hazard** - forward from MEM/WB to the EX stage:

```
1 else if (regwrite_memwb && (rd_memwb != 0) && (rd_memwb == rs1_idx))
2     forwardA = 2'b01;
3 else if (regwrite_memwb && (rd_memwb != 0) && (rd_memwb == rs2_idx))
4     forwardB = 2'b01;
```

The MEM hazard check performs the same comparison against the MEM/WB stage.

It is important to note that EX hazard forwarding takes priority over MEM hazard forwarding, which is handled naturally by the `if/else if` structure.

## 8.2 Hazard Detection Unit

### 8.2.1 Load-Use Data Hazard

Forwarding handles most RAW (Read-After-Write) hazards because the result of an arithmetic instruction is ready at the end of the EX stage and can be forwarded to the next instruction in time. The `ld` instruction is a special case: its result is not available until the end of the MEM stage, one cycle too late to be forwarded to the EX stage of the very next instruction. This gives `ld` a *two-cycle latency* for dependent instructions. The only way to handle this is to stall the pipeline for one cycle by inserting a bubble, allowing the loaded value to reach the MEM/WB register before it is consumed.

The following instruction pairs all trigger a stall:

1.

```
1 ld  x5, 0(x1)
2 sub x6, x5, x2
```

2.

```
1 ld  x5, 0(x1)
2 beq x5, x2, 0x8
```

3.

```
1 ld  x5, 0(x1)
2 sd  x2, 0(x5)
```

4.

```

1 ld x5, 0(x1)
2 ld x6, 0(x5)

```

To handle load-use data hazards, a separate **hazard detection unit** works alongside the forwarding unit. It checks whether the instruction currently in the ID/EX stage is a load (`memread_index = 1`) and whether its destination register `rd_index` conflicts with either source register (`rs1_id` or `rs2_id`) of the instruction in the IF/ID stage. When a conflict is detected, the stall signal is asserted, freezing the PC and the IF/ID register for one cycle while a NOP bubble is inserted into the ID/EX stage.

#### Inputs and Outputs:

##### • Inputs:

- `rs1_id[4:0]`, `rs2_id[4:0]` — source registers of the instruction in the IF/ID stage
- `rd_index[4:0]` — destination register of the instruction in the ID/EX stage
- `memread_index` — asserted when the ID/EX instruction is a load

##### • Output:

- `stall` — when asserted, freezes the PC and IF/ID register and inserts a NOP bubble into the ID/EX stage

The detection logic implemented in `hazard_detection_unit.v` is:

```

1 if (memread_index && (rd_index != 0) &&
2     ((rd_index == rs1_id) || (rd_index == rs2_id)))
3     stall = 1'b1;

```

This ensures that the loaded value has completed the MEM stage and is available in the MEM/WB register before the dependent instruction enters the EX stage, at which point the forwarding unit takes over and supplies the correct value.

### 8.2.2 Control Hazard

1. Static branch prediction in this design is implemented implicitly as always *NOT TAKEN*. The processor simply continues fetching sequential instructions unless the EX stage later confirms that the branch was actually taken.

```

1 assign flush = ex_branch_taken;
2 assign branch_taken = ex_branch_taken;

```

- `ex_branch_taken = 1` if branch is taken. This signal is generated only after the branch instruction reaches the EX stage.
- Until EX resolves the branch, the IF stage does not receive a taken-branch signal.
- Therefore, the fetch logic continues along the normal sequential path, i.e., `PC + 4`.

2. The IF stage receives the branch decision and branch target as inputs:

```

1 IF_stage IF(
2     .clk(clk),
3     .reset(reset),
4     .stall(stall),
5     .branch_taken(branch_taken),
6     .branch_target(ex_branch_target),
7     .pc(if_pc_current),
8     .raw_instruction(raw_instr)
9 );

```

- If `branch_taken = 0`, the IF stage keeps fetching the next sequential instruction.
- If `branch_taken = 1`, the IF stage redirects the PC to `ex_branch_target`, which is the target address for the branch.

- Since `branch_taken` comes only from EX, redirection to the correct path happens only after branch resolution.
- Thus, when the prediction is wrong, the instructions on the wrong-path must be flushed. To achieve this, we connect the flush signal to both the IF/ID and ID/EX pipeline registers.

## 9 Testing and Verification of Pipeline Processor

To verify the correctness of the pipelined processor, we tested the design using multiple testbenches. These included the provided reference testbench, a program generating the Fibonacci sequence, and a program computing the sum of the first n natural numbers. Note that the last 4 instructions, though not shown here, are dummy instructions that ensure that the last valid instruction can write back to the register file if required. The output of the register file for the provided testbench is shown below:

```
addi x2, x0, 5
addi x3, x0, 10
add x1, x2, x3
sub x2, x2, x3
and x4, x3, x3
and x5, x3, x4
or x6, x2, x4
or x7, x2, x3
sd x1, 0(x5)
ld x10, 0(x5)
sd x6, 24(x5)
ld x11, 24(x5)
beq x4, x5, 8
beq x0, x0, 0
add x13, x1, x10
```

Output in `register_file.txt`:

```
0000000000000000
000000000000000f
fffffffffffffffb
000000000000000a
000000000000000a
000000000000000a
fffffffffffffffb
fffffffffffffffb
0000000000000000
0000000000000000
000000000000000f
fffffffffffffffb
0000000000000000
000000000000001e
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
```



```

0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
21

```

## 9.1 Forwarding

```

addi x1, x0, 5
addi x2, x0, 7
add x3, x1, x2
sub x4, x3, x1
addi x5, x0, 9
addi x6, x0, 1
sub x7, x5, x6

```

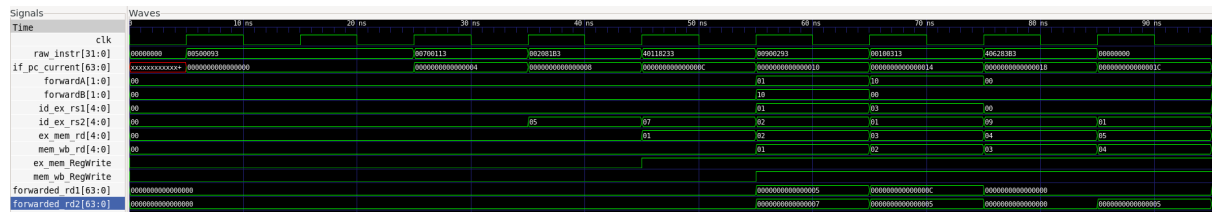


Figure 10: GTKWave showing forwarding

The waveform shows correct forwarding behavior for the dependent instructions.

- For the instruction `sub x4, x3, x1`, the value of register `x3` is produced by the previous instruction `add x3, x1, x2`. Since the value is still in the EX/MEM pipeline register, it is forwarded to the EX stage, which is reflected by `forwardA = 10` in the waveform.
- In the later instruction `sub x7, x5, x6`, two forwarding paths are used simultaneously. The value of `x6`, produced by the previous instruction, is forwarded from the EX/MEM stage (`forwardB = 10`), while the value of `x5`, produced two instructions earlier, is forwarded from the MEM/WB stage (`forwardA = 01`). This matches the expected EX and MEM hazard forwarding cases.

## 9.2 Stall

```

addi x1, x0, 16
lw x2, 0(x1)
add x3, x2, x2
add x4, x3, x2

```

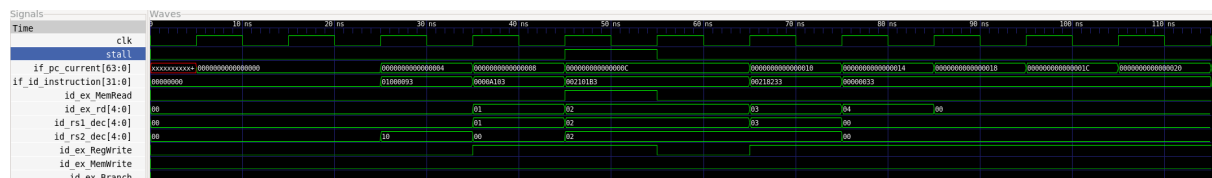


Figure 11: GTKWave showing stall

The waveform confirms correct load-use hazard handling.

- When the instruction `lw x2, 0(x1)` is followed immediately by `add x3, x2, x2`, the loaded value is not yet available at the beginning of the next cycle. Therefore, the hazard detection unit asserts `stall = 1`, preventing the PC and IF/ID register from advancing, which can be seen in the gtkwave.
- During the stall cycle, a NOP bubble is inserted into the ID/EX pipeline register by clearing its control signals. This delays the dependent instruction by one cycle, after which execution resumes normally and the required value can be used safely.

### 9.3 Flush

```
addi x1, x0, 5
addi x2, x0, 5
beq x1, x2, 8
addi x3, x0, 9
addi x4, x0, 7
```

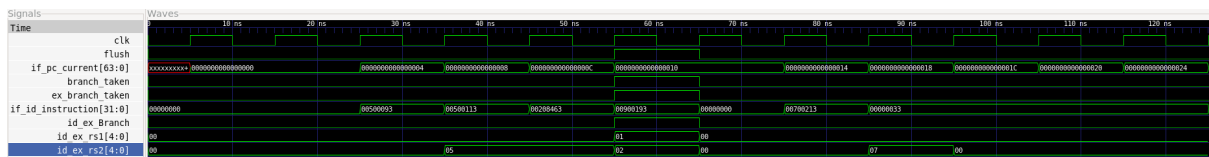


Figure 12: GTKWave showing flush

The waveform confirms correct flush behavior for a taken branch.

- When `beq x1, x2, 8` is resolved in the EX stage and the branch condition is satisfied, `ex_branch_taken` is set to 1, thus `branch_taken` and `flush` are asserted, causing the sequentially fetched wrong-path instruction `addi x3, x0, 9` to be discarded.
- The program counter is then redirected to the branch target, and execution resumes from `addi x4, x0, 7`. As a result, `x3` remains unchanged, while `x4` is updated correctly, confirming successful pipeline flushing.
- The branch flush penalty in this design is 2 cycles, since the branch outcome is resolved only in the EX stage. By that time, the next two instructions have already entered the pipeline (in IF and ID), and both must be discarded when the branch is taken.

## 10 Contributions

### 10.1 Sequential

- Dhanika – Implementation of Control unit, ALU Control, Register file.
- Moksha – Implementation of Data memory, Immediate Generation, Mux
- Gauri – Implementation of PC, Instruction memory and Adder

Gauri's ALU module was implemented in the processor. The final integration of the Sequential Processor was done by both Moksha and Gauri.

### 10.2 Pipelining

The datapath modules used in the pipelined design are the same as those implemented in the sequential processor.

- Dhanika – Implementation of the pipeline stages and pipeline registers.
- Moksha – Implementation of the forwarding and hazard detection unit.

- Gauri – Final integration of all modules, along with debugging and verification of the complete pipelined system.

The report was prepared collaboratively by all three members.